

SHOPSY E-COMMERCE DATABASE MANAGEMENT SYSTEM

REVIEW AND RATING MANAGEMENT SYSTEM

INTRODUCTION :

A Product Review and Rating Management System is used to store and manage customer feedback about products. It helps customers provide reviews and ratings based on their experience with a product. In this system, the Review table stores customer reviews and the Rating table stores the ratings given for each review. SQL queries and aggregate functions such as AVG() can be used to calculate the average rating of products and identify highly rated products. This system helps in organizing customer feedback and analyzing product performance effectively.

OBLECTIVE :

- To create Review and Rating tables using DBMS concepts.
- To store customer feedback and product ratings.
- To retrieve product review details using SQL queries.
- To calculate average product ratings using aggregate functions.
- To identify highly rated products based on their average ratings.
- To manage customer reviews and ratings in an organized database system.

3. Review Table :

Attribute	Key Type
Review_ID	Primary Key
Customer_ID	Foreign Key

Attribute	Key Type
Product_ID	Foreign Key
Review_Text	-
Review_Date	-

4. Rating Table :

Attribute	Key Type
Rating_ID	Primary Key
Customer_ID	Foreign Key
Product_ID	Foreign Key
Rating	-
Rating_Date	-

5.Create Review Table :

```
SQL> CREATE TABLE Review (
2     Review_ID NUMBER PRIMARY KEY,
3     Customer_ID NUMBER,
4     Product_ID NUMBER,
5     Review_Text VARCHAR2(500),
6     Review_Date DATE
7 );
```

Table created.

6.Create Rating Table :

```
SQL> CREATE TABLE Rating (  
2     Rating_ID INT PRIMARY KEY,  
3     Customer_ID INT,  
4     Product_ID INT,  
5     Rating INT NOT NULL CHECK (Rating BETWEEN 1 AND 5),  
6     Rating_Date DATE NOT NULL,  
7     FOREIGN KEY (Customer_ID) REFERENCES Customer(ID),  
8     FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)  
9 );
```

Table created.

7.Review Data Insertion :

```
SQL> INSERT INTO Review VALUES  
2 (1, 101, 201, 'Good product quality', DATE '2026-10-01');  
  
1 row created.
```

```
SQL> INSERT INTO Review VALUES  
2 (2, 102, 201, 'Worth the price', DATE '2026-10-02');  
  
1 row created.
```

```
SQL> INSERT INTO Review VALUES  
2 (3, 103, 202, 'Excellent product', DATE '2026-10-03');  
  
1 row created.
```

```
SQL> INSERT INTO Review VALUES  
2 (4, 104, 203, 'Average quality', DATE '2026-10-04');  
  
1 row created.
```

```
SQL> INSERT INTO Review VALUES  
2 (5, 105, 202, 'Very useful product', DATE '2026-10-05');  
  
1 row created.
```

```
SQL> INSERT INTO Review VALUES  
2 (6, 106, 204, 'Good quality product', DATE '2026-10-06');  
  
1 row created.
```

```
SQL> INSERT INTO Review VALUES  
2 (7, 107, 205, 'Excellent quality', DATE '2026-10-07');  
  
1 row created.
```

8. Rating Data Insertion :

```
SQL> INSERT INTO Rating VALUES
  2  (1, 1, 101, 5, DATE '2026-10-01');

1 row created.

SQL> INSERT INTO Rating VALUES
  2  (2, 2, 102, 4, DATE '2026-10-02');

1 row created.

SQL> INSERT INTO Rating VALUES
  2  (3, 3, 103, 5, DATE '2026-10-03');

1 row created.

SQL> INSERT INTO Rating VALUES
  2  (4, 4, 104, 3, DATE '2026-10-04');

1 row created.

SQL> INSERT INTO Rating VALUES
  2  (5, 5, 105, 4, DATE '2026-10-05');

1 row created.

SQL> INSERT INTO Rating VALUES
  2  (6, 6, 106, 5, DATE '2026-10-06');

1 row created.

SQL> INSERT INTO Rating VALUES
  2  (7, 2, 105, 4, DATE '2026-10-07');

1 row created.
```

9. Retrieve Product Review Details :

```
SQL> SELECT
  2     R.RATING_ID,
  3     C.NAME AS CUSTOMER_NAME,
  4     P.PRODUCT_ID,
  5     P.PRODUCT_NAME,
  6     R.RATING,
  7     R.RATING_DATE
  8 FROM Rating R
  9 JOIN Customer C
 10     ON R.CUSTOMER_ID = C.ID
 11 JOIN Product P
 12     ON R.PRODUCT_ID = P.PRODUCT_ID
 13 ORDER BY R.RATING_ID;
```

RATING_ID	CUSTOMER_NAME	PRODUCT_ID		
PRODUCT_NAME			RATING	RATING_DA
Laptop	1 Ramesh	101	5	01-OCT-26
Smartphone	2 Khilan	102	4	02-OCT-26
Headphones	3 Kaushik	103	5	03-OCT-26
RATING_ID	CUSTOMER_NAME	PRODUCT_ID		
PRODUCT_NAME			RATING	RATING_DA
Keyboard	4 Chaitali	104	3	04-OCT-26
Mouse	5 Hardik	105	4	05-OCT-26
Monitor	6 Komal	106	5	06-OCT-26
RATING_ID	CUSTOMER_NAME	PRODUCT_ID		
PRODUCT_NAME			RATING	RATING_DA
Mouse	7 Khilan	105	4	07-OCT-26

7 rows selected.

10.Calculate Average Product Rating :

```
SQL> SELECT
  2     PRODUCT_ID,
  3     ROUND(AVG(RATING), 2) AS AVERAGE_RATING,
  4     COUNT(RATING_ID) AS TOTAL_RATING
  5 FROM Rating
  6 GROUP BY PRODUCT_ID
  7 ORDER BY PRODUCT_ID;
```

PRODUCT_ID	AVERAGE_RATING	TOTAL_RATING
101	5	1
102	4	1
103	5	1
104	3	1
105	4	2
106	5	1

6 rows selected.

11. Identify Highly Rated Product :

```
SQL> SELECT
  2     P.PRODUCT_ID,
  3     P.PRODUCT_NAME,
  4     ROUND(AVG(R.RATING), 2) AS AVERAGE_RATING
  5 FROM Product P
  6 JOIN Rating R
  7 ON P.PRODUCT_ID = R.PRODUCT_ID
  8 GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  9 HAVING AVG(R.RATING) >= 4
 10 ORDER BY AVERAGE_RATING DESC;
```

PRODUCT_ID	PRODUCT_NAME	AVERAGE_RATING
101	Laptop	5
103	Headphones	5
106	Monitor	5
105	Mouse	4
102	Smartphone	4

12. Product Rating Report :

```
SQL> SELECT
  2     P.PRODUCT_ID,
  3     P.PRODUCT_NAME,
  4     ROUND(AVG(R.RATING), 2) AS AVERAGE_RATING,
  5     COUNT(R.RATING_ID) AS TOTAL_RATINGS
  6 FROM Product P
  7 JOIN Rating R
  8 ON P.PRODUCT_ID = R.PRODUCT_ID
  9 GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
 10 ORDER BY P.PRODUCT_ID;
```

PRODUCT_ID	PRODUCT_NAME	AVERAGE_RATING
TOTAL_RATINGS		
101	Laptop 1	5
102	Smartphone 1	4
103	Headphones 1	5

PRODUCT_ID	PRODUCT_NAME	AVERAGE_RATING
TOTAL_RATINGS		
104	Keyboard 1	3
105	Mouse 2	4
106	Monitor 1	5

6 rows selected.